

Introduction à **Python**

Un cours condensé pour apprendre les bases du langage Python : syntaxe, structures de données, fonctions, programmation orientée objet et bonnes pratiques.



Python
3



pip &
venv

1 Qu'est-ce que Python ?

Comprendre le langage et préparer son environnement de travail.

Un langage simple et polyvalent

Python est un langage interprété, au typage dynamique, connu pour sa syntaxe lisible proche du pseudo-code. Il est utilisé aussi bien pour le développement web, l'automatisation, la data science, l'intelligence artificielle que le scripting système.

Syntaxe claire

Typage dynamique

Immense écosystème

Multi-plateforme

Installer son environnement

- 1 Télécharger **Python 3** depuis python.org (cocher "Add to PATH" sous Windows).
- 2 Installer un éditeur : **VS Code** avec l'extension Python, ou PyCharm.
- 3 Vérifier l'installation avec `python --version` dans un terminal.
- 4 Créer un environnement virtuel : `python -m venv env`, puis l'activer.

Premier script

```
$ python3 hello.py Bonjour, Python !
```

```
# hello.py
print("Bonjour, Python !")
```

ASTUCE

L'interpréteur interactif (python3 sans argument, ou IPython) permet de tester des instructions ligne par ligne — idéal pour explorer le langage.

2 Variables, types & opérateurs

Pas de point-virgule, l'indentation structure le code.

Variables & types simples

```
# Pas besoin de déclarer le type
age = 25          # int
prix = 9.99       # float
nom = "Alice"     # str
actif = True      # bool
rien = None       # NoneType

type(age) # <class 'int'>
```

f-strings & conversions

```
# Formatage de chaînes
nom = "Sam"
age = 30
msg = f"{nom} a {age} ans"

# Conversion de type
texte = str(42)
nombre = int("42")
flottant = float("3.14")
```

Opérateurs courants

Catégorie	Opérateurs	Exemple
Arithmétiques	+ - * / // % **	7 // 2 → 3, 2 ** 3 → 8
Comparaison	== != < > <= >=	5 > 3 → True
Logiques	and or not	True and False → False

Structures de données intégrées

```
liste = [1, 2, 3]
liste.append(4)

tuple_ = (1, "a", True) # immuable

ensemble = {1, 2, 2, 3} # {1,2,3}
```

```
personne = {
    "nom": "Léa",
    "age": 28,
}
print(personne["nom"])
personne["ville"] = "Lyon"
```

3 Conditions & boucles

L'indentation (4 espaces) délimite les blocs — pas d'accolades.

Conditions

```
note = 14

if note >= 16:
    print("Excellent")
elif note >= 10:
    print("Admis")
else:
    print("Recalé")
```

Boucle for

```
for fruit in ["pomme", "kiwi"]:
    print(fruit)

for i in range(5):
    print(i)  # 0 à 4

for i, v in enumerate(liste):
    print(i, v)
```

Boucle while

```
compteur = 0
while compteur < 3:
    print(compteur)
    compteur += 1
else:
    print("Terminé")
```

break / continue

```
for n in range(10):
    if n == 5:
        break
    if n % 2 == 0:
        continue
    print(n)
```

Compréhensions de liste

```
carres = [x**2 for x in range(10)]
pairs = [x for x in range(20) if x % 2 == 0]
carres_dict = {x: x**2 for x in range(5)}
```

ASTUCE

Les compréhensions remplacent avantageusement les boucles for qui construisent une liste : plus concises et souvent plus rapides.

4 Organiser son code

Découper la logique en fonctions réutilisables, puis en modules.

Déclarer une fonction

```
def addition(a, b):  
    return a + b  
  
# Valeur par défaut  
def saluer(nom="ami"):  
    print(f"Bonjour {nom}")  
  
saluer()  
saluer("Nadia")
```

```
# Nombre variable d'arguments  
def total(*nombres):  
    return sum(nombres)  
  
total(1, 2, 3) # 6  
  
# Arguments nommés multiples  
def profil(**infos):  
    return infos
```

Fonctions lambda

```
carre = lambda x: x ** 2  
tri_par_age = sorted(personnes, key=lambda p: p["age"])
```

Modules & packages

```
# math_utils.py  
def double(x):  
    return x * 2  
  
# main.py  
import math_utils  
from math_utils import double
```

```
# Modules de la bibliothèque standard  
import math  
import random  
import datetime as dt  
  
math.sqrt(16) # 4.0  
random.randint(1, 10)
```

GESTION DES DÉPENDANCES

`pip install nom_package` installe une bibliothèque tierce. Le fichier `requirements.txt` liste les dépendances d'un projet pour les réinstaller ailleurs avec `pip install -r requirements.txt`.

5 Classes & objets

Modéliser des entités avec leurs propres données et comportements.

Déclarer une classe

```
class Animal:
    def __init__(self, nom, age):
        self.nom = nom
        self.age = age

    def crier(self):
        print(f"{self.nom} fait du bruit")

class Chien(Animal):    # héritage
    def crier(self):    # redéfinition (override)
        print(f"{self.nom} aboie !")

rex = Chien("Rex", 3)
rex.crier() # Rex aboie !
```

CONCEPTS CLÉS

- **self** représente l'instance courante, toujours premier paramètre des méthodes
- **__init__** est le constructeur, appelé à la création de l'objet
- **Attributs de classe** vs **attributs d'instance**, **méthodes statiques** (@staticmethod)

Propriétés & encapsulation

```
class CompteBancaire:
    def __init__(self, solde=0):
        self._solde = solde # convention "privé"

    @property
    def solde(self):
        return self._solde

    def déposer(self, montant):
        self._solde += montant
```

POURQUOI LA POO ?

Elle permet de regrouper données et comportements associés, de réutiliser du code par héritage, et de modéliser des systèmes complexes de façon plus lisible.

6 Gérer les erreurs & les fichiers

Rendre son code robuste face aux imprévus et lire/écrire des données.

Try / except

```
try:
    resultat = 10 / 0
except ZeroDivisionError:
    print("Division par zéro impossible")
except Exception as e:
    print(f"Erreur inattendue : {e}")
else:
    print("Aucune erreur")
finally:
    print("Toujours exécuté")
```

Lever ses propres exceptions

```
def valider_age(age):
    if age < 0:
        raise ValueError("L'âge ne peut pas être négatif")
    return age
```

Lire & écrire des fichiers

```
# Écriture
with open("notes.txt", "w") as f:
    f.write("Bonjour\n")

# Lecture
with open("notes.txt") as f:
    contenu = f.read()
```

```
# Lire un fichier JSON
import json
with open("data.json") as f:
    data = json.load(f)

with open("out.json", "w") as f:
    json.dump(data, f)
```

ASTUCE

Le mot-clé `with` (context manager) ferme automatiquement le fichier, même en cas d'erreur — préférez-le toujours à un `open()/close()` manuel.

7 Bibliothèques & environnements

Tirer parti de l'écosystème sans réinventer la roue.

Quelques modules de la bibliothèque standard

Module	Usage
os / pathlib	Manipuler fichiers, dossiers et chemins
datetime	Dates et heures
re	Expressions régulières
collections	Structures avancées (Counter, defaultdict...)
unittest / pytest	Tests automatisés

Environnements virtuels

```
$ python -m venv env $ source env/bin/activate # macOS/Linux $ env\Scripts\activate # Windows (env) $ pip install requests (env) $ pip freeze > requirements.txt
```

POURQUOI ISOLER SES PROJETS ?

Un environnement virtuel isole les dépendances d'un projet du reste du système, évitant les conflits de versions entre plusieurs projets Python.

Bibliothèques tierces incontournables

Domaine	Packages populaires
Web	Flask, Django, FastAPI
Data / Calcul scientifique	NumPy, pandas, Matplotlib
Requêtes HTTP	requests, httpx
Machine learning	scikit-learn, PyTorch, TensorFlow

8 Bonnes pratiques & ressources

Les réflexes à adopter pour écrire du code Python propre et maintenable.

Style & lisibilité

- Suivre la convention **PEP 8** (nommage, indentation, longueur de ligne)
- Nommer variables et fonctions en `snake_case`
- Ajouter des *docstrings* pour documenter fonctions et classes

Tests

- **Tests unitaires** avec `pytest` pour chaque fonction critique
- Nommer les fichiers `test_*.py` pour la découverte automatique
- Viser une couverture de test suffisante sur la logique métier

Outils de qualité

- `black` pour le formatage automatique
- `flake8` / `ruff` pour l'analyse statique (linting)
- `mypy` pour vérifier les annotations de type

Annotations de type

```
def addition(a: int, b: int) -> int:  
    return a + b
```

Prochaines étapes

Vous avez maintenant les bases pour écrire vos premiers programmes. Continuez avec :

- La documentation officielle : **docs.python.org**
- Le tutoriel interactif : **python.org/about/gettingstarted**
- PyPI pour explorer les packages : **pypi.org**
- Construire un petit projet complet : gestionnaire de tâches, scraper web, ou jeu simple